Research Paper / Internal Knowledge Base Search

DBMS Mini Project Phase-1 Report

1. Abstract

This mini project implements a research-paper internal knowledge base using PostgreSQL and the pgvector extension. The database stores users, research papers, authors, tags, citation relationships, and paper chunks. Papers are divided into smaller chunks so that each passage can be represented by a vector embedding. The embedding column uses pgvector's vector(1536) type, while an HNSW index supports efficient approximate nearest-neighbor search using cosine distance. The design also demonstrates relational integrity through primary keys, foreign keys, unique constraints, check constraints, cascading actions, views, and an update timestamp trigger. The current SQL includes demo random vectors for testing the vector infrastructure; a production deployment would replace them with embeddings generated by an actual embedding model.

2. Introduction

The Research Paper / Internal Knowledge Base Search project is a database-oriented system for organizing and retrieving research literature. It combines conventional relational database design with vector-based semantic-search infrastructure. PostgreSQL provides the core database, while pgvector adds vector storage and similarity-search operations.

The database is designed around research papers and their related entities. Each paper can have multiple authors, tags, citations, and content chunks. Chunk-level embeddings make it possible to retrieve passages according to semantic similarity rather than relying only on exact keyword matching.

3. Objectives

- Design a normalized relational database for an internal research-paper knowledge base.
- Store paper metadata, users, authors, tags, citation relationships, and document chunks.
- Support semantic-search infrastructure by storing 1536-dimensional embeddings using pgvector.
- Use HNSW indexing for efficient vector similarity search with cosine distance.
- Demonstrate database integrity using primary keys, foreign keys, unique/check constraints, and referential actions.
- Provide reusable views for paper details and citation-network information.
- Automatically maintain the paper updated_at timestamp using a PostgreSQL trigger.

4. Technologies and Dependencies

Component	Role
PostgreSQL	Primary relational database; stores all project entities and relationships.
pgvector	PostgreSQL extension providing the VECTOR data type and vector similarity operators.
HNSW index	Approximate nearest-neighbor index on the embedding column for faster similarity retrieval.
PL/pgSQL	Used for the update_paper_timestamp trigger function.

Embedding model (application layer)	Required in a complete semantic-search application to convert query/chunk text into meaningful vectors. The submitted SQL does not include an embedding model.
SQL / psql	Used to create the database, tables, constraints, indexes, views, trigger, sample data, and test queries.

5. Basic Functionalities

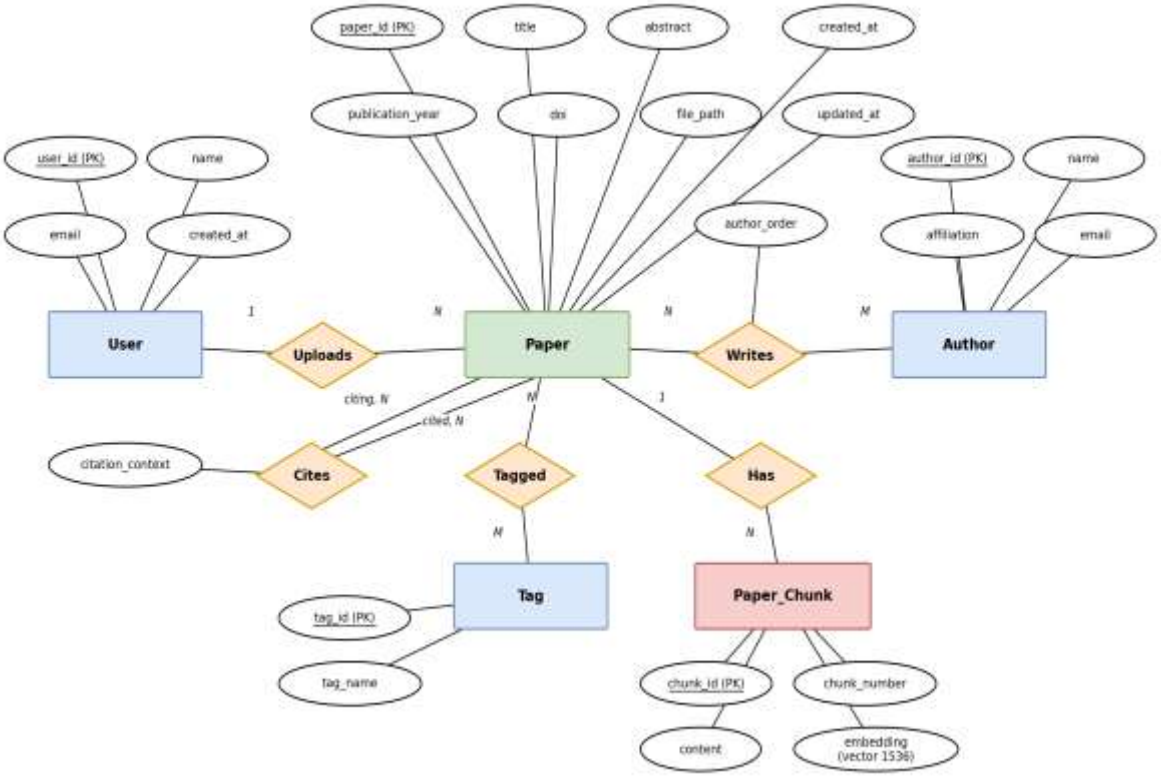
Functionality	Description
Paper management	Stores title, abstract, year, DOI, file path, uploader, and timestamps.
Author management	Stores authors and supports multiple authors per paper while preserving author order.
Tagging	Allows papers to be categorized with reusable tags.
Citation tracking	Stores directed paper-to-paper citations and optional citation context.
Chunking	Stores smaller passages of papers rather than requiring one embedding for the entire document.
Vector storage	Stores one 1536-dimensional vector per chunk using pgvector.
Semantic similarity search	Compares vectors using cosine distance and returns the closest chunks.
Performance indexing	Uses HNSW for vector search and B-tree indexing on paper_id for chunk lookups.
Views	paper_details and citation_network simplify common joins.
Automatic timestamps	A BEFORE UPDATE trigger maintains papers.updated_at.

6. System Workflow

1. User interacts with the Knowledge Base Search system.
2. Paper information and document content are stored in the database.
3. Papers are divided into smaller chunks for processing.
4. Each paper chunk is assigned an embedding for semantic search.
5. Chunks and their embeddings are stored in the paper_chunks table.
6. A search query is processed and compared with stored embeddings.
7. Relevant paper chunks are retrieved and linked back to their papers.
8. The system presents the relevant research-paper information to the user.

7. ER Diagram

The ER model below reflects the actual implemented tables and relationships. Paper–Author and Paper–Tag are many-to-many relationships resolved using junction tables. Citations form a self-referencing relationship on the papers table, while each paper can contain many chunks.



8. Relational Schema:

USERS **(**user_id, name, email, created_at)
PAPERS **(**paper_id, title, abstract, publication_year, doi, file_path, uploaded_by, created_at, updated_at)
AUTHORS **(**author_id, name, affiliation, email)
PAPER_AUTHORS **(**paper_id, author_id, author_order)
TAGS **(**tag_id, tag_name)
PAPER_TAGS **(**paper_id, tag_id)
PAPER_CITATIONS **(**citing_paper_id, cited_paper_id, citation_context, created_at)
PAPER_CHUNKS **(**chunk_id, paper_id, chunk_number, content, embedding, created_at)

9. Table-wise Description

Table	Attributes / Keys	Purpose
users	user_id PK; name; email UNIQUE; created_at	Stores users who upload papers.
papers	paper_id PK; title; abstract; publication_year; doi UNIQUE; file_path; uploaded_by FK; created_at; updated_at	Stores core research-paper metadata.
authors	author_id PK; name; affiliation; email UNIQUE	Stores author information.
paper_authors	paper_id PK/FK; author_id PK/FK; author_order CHECK > 0	Junction table for the Paper–Author many-to-many relationship.
tags	tag_id PK; tag_name UNIQUE	Stores reusable paper tags.
paper_tags	paper_id PK/FK; tag_id PK/FK	Junction table for the Paper–Tag many-to-many relationship.

paper_citations	citing_paper_id PK/FK; cited_paper_id PK/FK; citation_context; created_at; no-self-citation CHECK	Models directed citations between papers.
paper_chunks	chunk_id PK; paper_id FK; chunk_number; content; embedding vector(1536); created_at; UNIQUE(paper_id, chunk_number)	Stores chunked paper content and vector embeddings.

10. Database Design Details

• Papers are split into chunks (paper_chunks) rather than embedded as a whole document — each chunk gets its own embedding, which is standard practice for semantic search over long text and lets retrieval surface the specific passage that matched, not just the paper.
• An HNSW index (idx_paper_chunks_embedding_hnsw) is built on the embedding column for fast approximate nearest-neighbor similarity search, alongside a standard B-tree index on paper_id for chunk lookups.
• paper_details and citation_network are views that pre-join commonly queried data (uploader name, citing/cited paper titles) to simplify application-side queries.
• A BEFORE UPDATE trigger (trg_update_paper_timestamp) automatically maintains papers.updated_at, removing the need for the application layer to set it manually.
• The paper_authors junction table includes author_order (with a CHECK > 0) to preserve the original author sequence, and paper_citations includes citation_context to record why one paper cites another.

11. Important Implementation Note

The current SQL creates demonstration embeddings using PostgreSQL random-number generation. Therefore the stored vectors are not semantic representations of the sample text. The vector schema, pgvector operators, and HNSW index are implemented, but a production semantic-search application must generate embeddings using an actual embedding model and insert those vectors into paper_chunks.embedding.

12. Sample Semantic Search Logic

1. Accept a natural-language query from the user.

2. Generate a query embedding using the same embedding model used for document chunks.

3. Compare the query embedding with paper_chunks.embedding using cosine distance.

4. Order the results by the smallest distance.

5. Return the most relevant chunks together with their paper metadata.

The submitted SQL demonstrates the vector comparison using the pgvector cosine-distance operator (<=>).

13. Limitations and Future Enhancements

- The current SQL uses random 1536-dimensional demo vectors. These are suitable for testing the vector infrastructure but are not meaningful semantic embeddings.
- No user authentication or role-based access control is implemented in the submitted users table; there is no password_hash or role column.
- PDF extraction and automatic chunking are not implemented in the submitted SQL.

- The application-layer code that converts a natural-language query into an embedding is not included in the submitted SQL.
- Production deployment would require appropriate access control, backup/recovery, input validation, and a real embedding model.

14. Conclusion

The project demonstrates how a conventional relational database can be extended for semantic-search workloads without introducing a separate vector database. PostgreSQL remains the single database, while pgvector supplies vector capabilities. The schema provides structured storage for research-paper metadata and relationships, and the chunk-level embedding design establishes the foundation for semantic retrieval.